A Local AI Programming Assistant

Small, coursework-adapted models on a 6 GB student laptop

Arpita Ghadai (B523010) · Arpita Nayak (B523011) · Manish Biswal (B523035)

Supervisor: Prof. Anjali Mohapatra · Evaluation Panel 2 · Technical Writing, November 2026

Students pay for online AI coding help

It costs money

Subscriptions every month, for the whole semester.

Code leaves your laptop

Every question sends the student's code to someone else's server.

It needs the internet

No connection in the hostel or lab means no help.

Our idea: run a small AI model on the student's own laptop, taught our programming courses, and check every answer by running tests.

Our research question

“Can a small AI model, taught our programming courses, give correct programming help on a 6 GB laptop — fast enough and cheap enough to be useful?”

Quality How many exercises does it solve correctly?

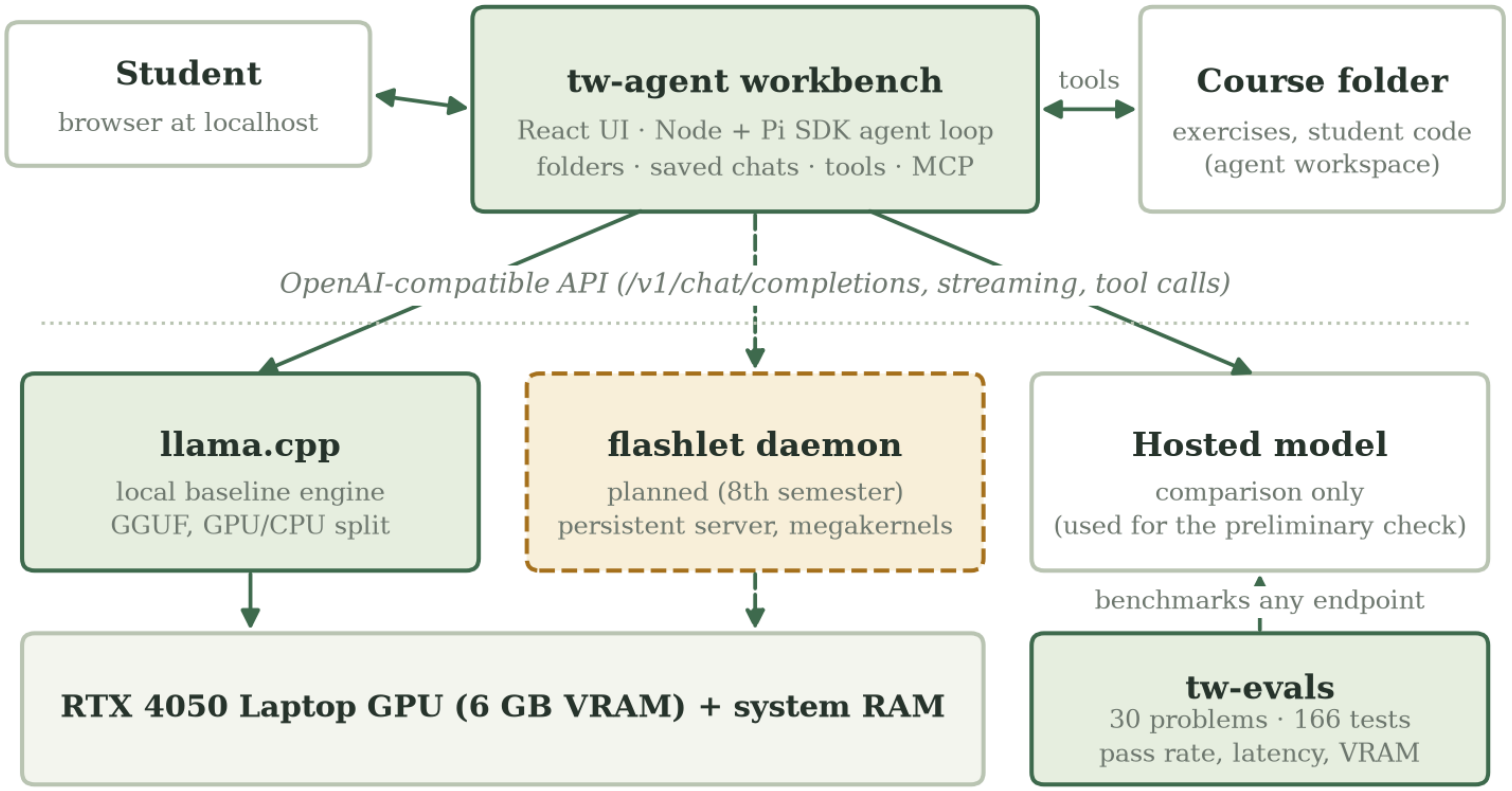
Fit Which models fit in 6 GB of GPU memory?

Speed How quickly does the answer appear?

Teaching Does training on our courses make it better?

Cost What does one exercise cost, local vs paid?

The whole system in one picture



Student app

tw-agent: chat with an AI that can read, write and run code in your folder

Engine

the program that runs the model on the laptop GPU

Examiner

tw-evals: 30 problems with tests that grade the code automatically

How it works

What we built · the 6 GB problem · mixture of experts · offloading · megakernels · teaching the model

Built first: the flashlet inference engine

Manish Biswal · Tech

112.5	5,961	140	99.3%
words/s for one user	words/s with 128 users	words/s with our megakernel	match with the reference model

Continuous batching

New requests join while others are still being answered

Paged memory

Conversation memory handed out in small pages, like an operating system does

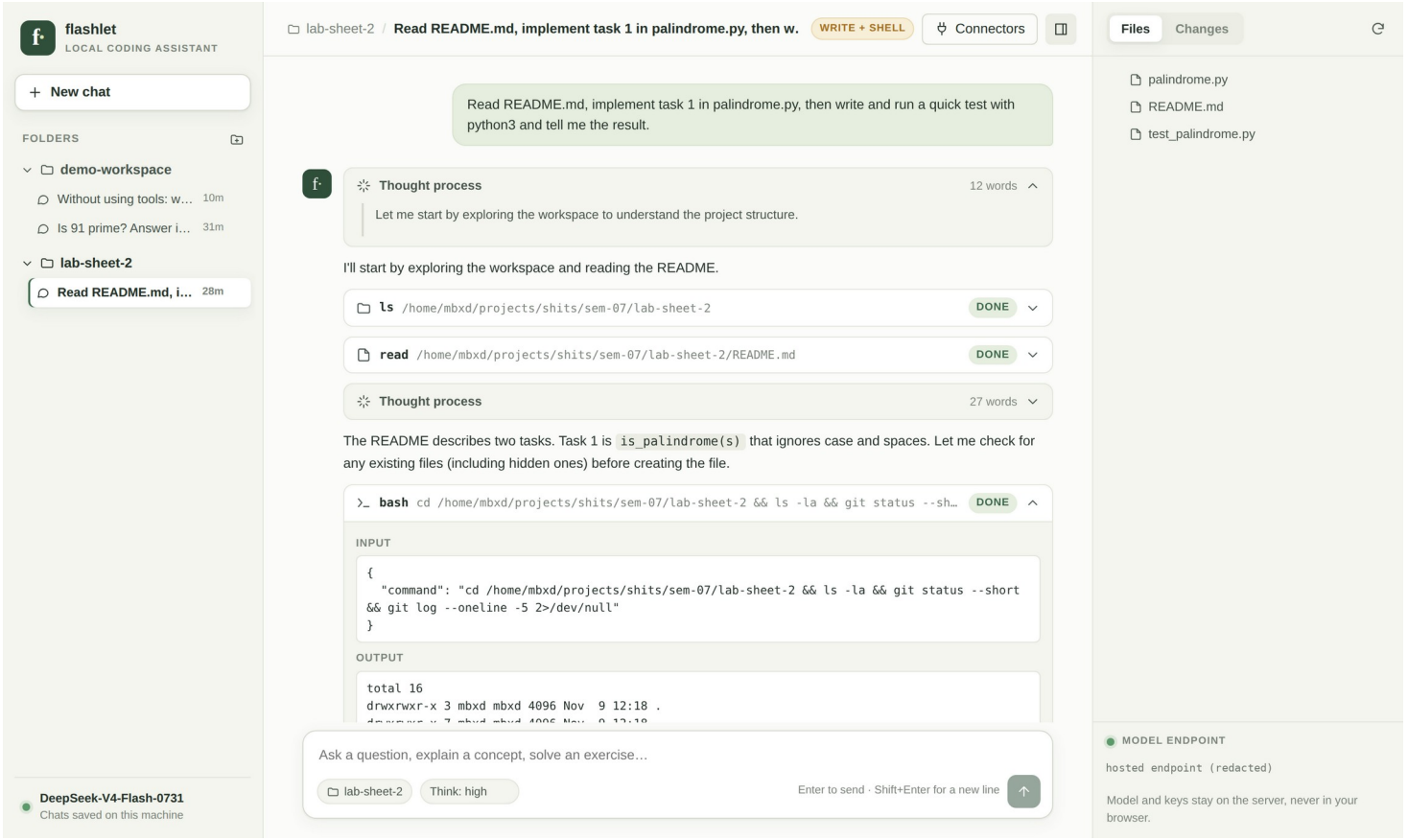
Megakernel prototype

Whole model in one GPU program: the idea we scale up next

Measured on our RTX 4050 laptop GPU with a small 0.5B model (Qwen2.5-0.5B). Built this semester, before tw-agent and tw-evals.

Built: the tw-agent workbench

Manish Biswal · Tech



Folders & saved chats

Each lab sheet is a folder; chats survive restarts

Visible thinking

The model's reasoning streams live

Tool cards

Every file read, write and command it runs

Changes view

A diff of every edit to your files

Built: an examiner that runs the code

Manish Biswal · Tech

30

practice problems

166

executable tests

166/166

reference solutions pass

First end-to-end check (hosted model, testing the pipeline)

Problem 021 — prime factors

Agent wrote the solution and its own check script

6/6

tests, confirmed by tw-evals

Lab sheet — palindrome

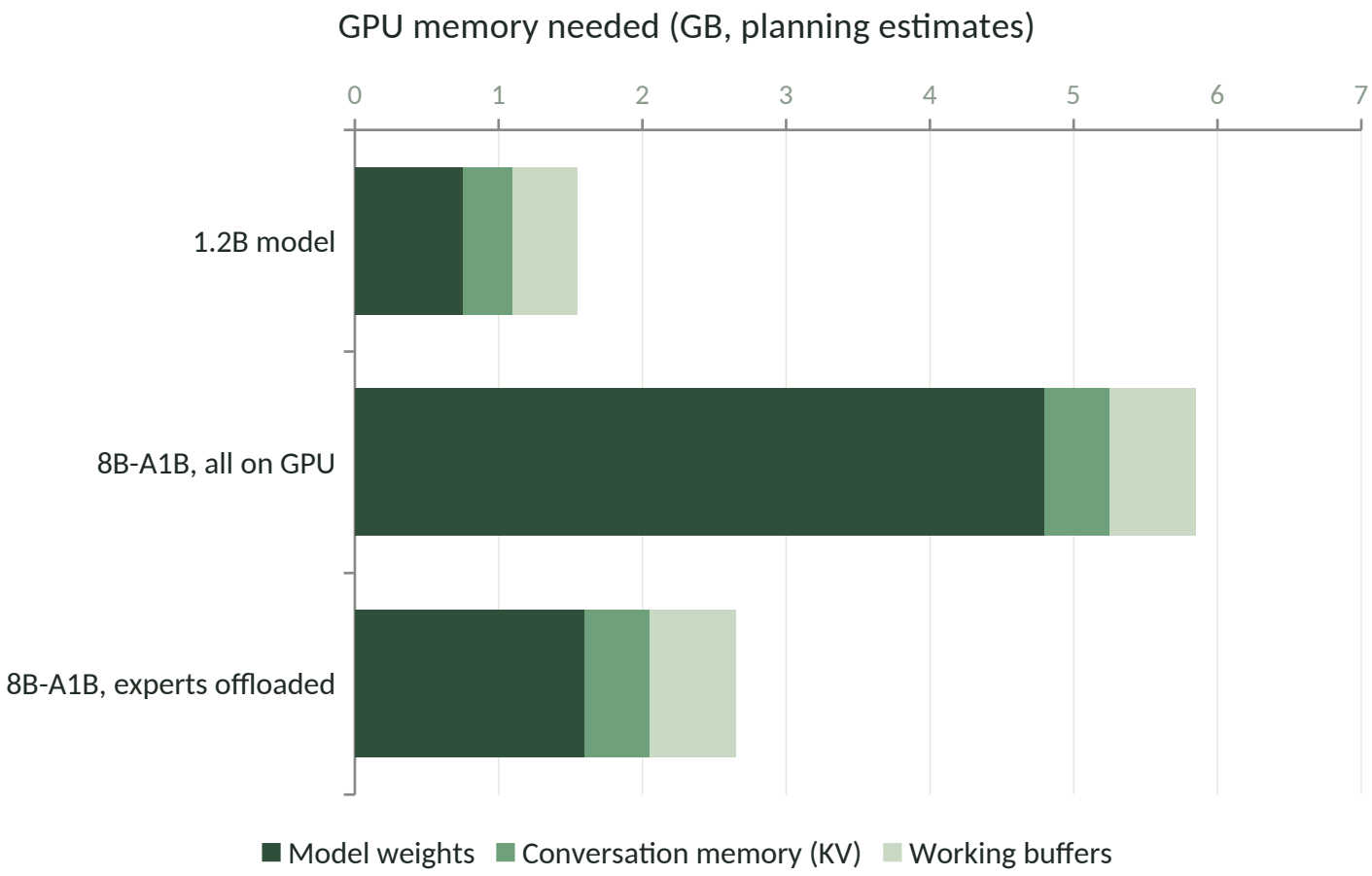
Read the README, wrote palindrome.py and a test, ran it

10/10

of the agent's tests

The catch: only 6 GB of GPU memory

Manish Biswal · Tech



What uses GPU memory

Weights: the model's knowledge, billions of numbers

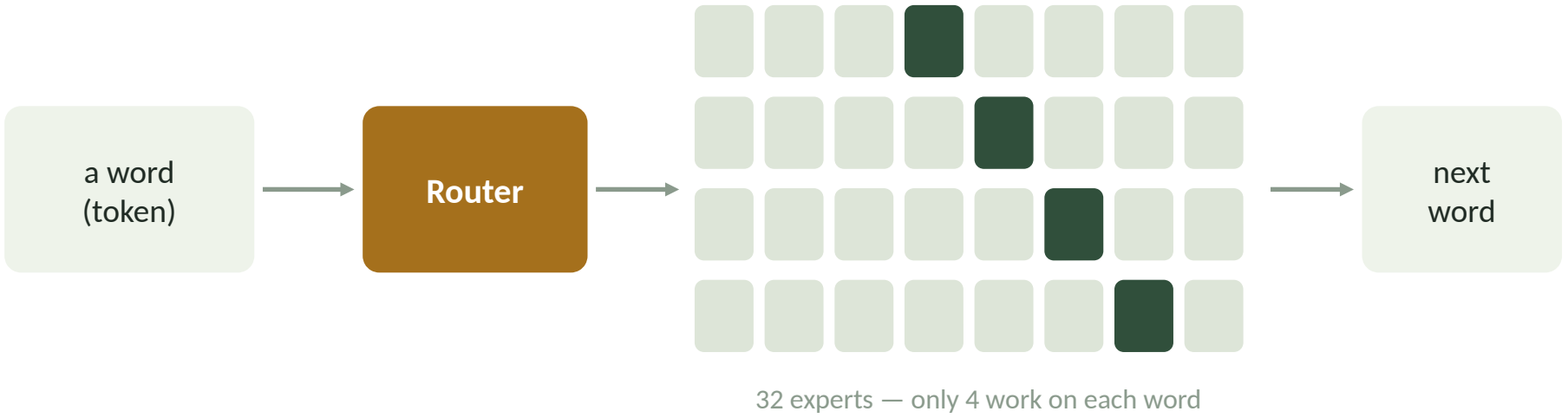
Conversation memory: grows with longer chats

Buffers: scratch space while computing

The 8B model barely fits (5.9 of 6 GB). Moving rarely-used parts to normal RAM frees most of the GPU.

What is a Mixture of Experts (MoE)?

Manish Biswal · Tech



32 experts — only 4 work on each word

8.3 B

numbers stored

1.5 B

used per word

Analogy: a hospital with 32 specialists. A receptionist (the router) sends each patient (word) to the 4 doctors who fit best. The hospital is big, but each visit is quick.

Offloading experts to normal RAM

Manish Biswal · Tech

GPU memory (6 GB)

Fast: about 192 GB/s

- Shared layers (always needed)
- Expert cache: recently used experts
- Conversation memory

PCIe link
≈ 16 GB/s — 12× slower

System RAM

Large but far away

all other experts

1 Load on demand

Router asks for an expert not on the GPU → copy it in

2 Keep a cache

Recently used experts stay; the oldest is evicted (LRU)

3 Guess ahead

Predict the next experts and copy them early, then measure if it helps

Speed is limited by memory, not maths

Manish Biswal · Tech

words per second ≤ **memory speed ÷ bytes read per word**

192 GB/s

memory speed of our RTX 4050 laptop GPU

≈ 194

words/s ceiling for a 0.5B model (1 GB to read)

140

words/s our megakernel prototype reached

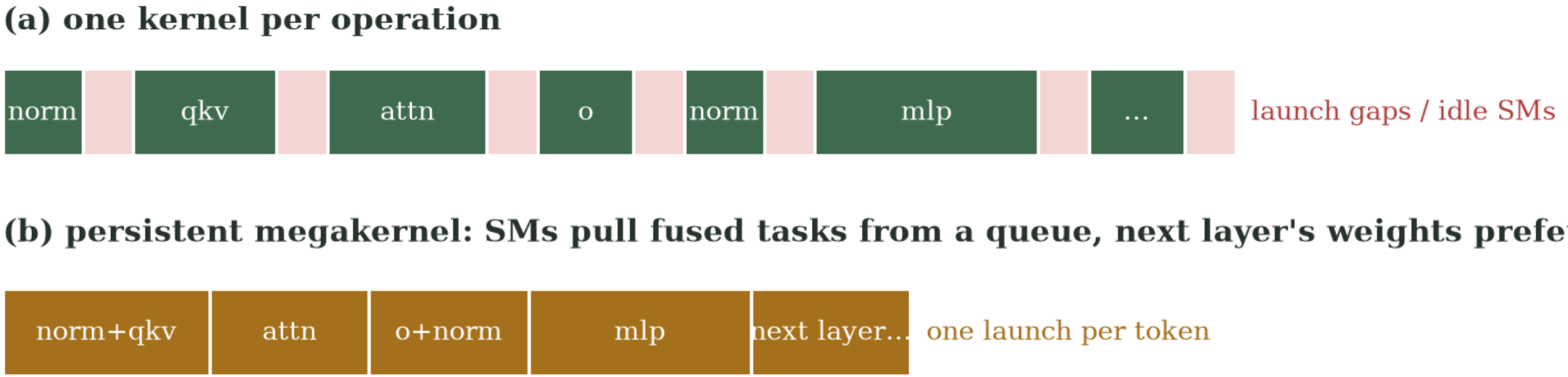
72%

of the ceiling already used

In simple words: to write each new word the GPU must re-read every active weight once. The faster it can read memory, the faster it can write. So we try to read less (small, quantised models, MoE) and never leave the GPU idle.

What is a megakernel?

Manish Biswal · Tech



Kernel

one small program the GPU runs, e.g. “multiply these matrices”

Today

~dozens of kernels per layer, with idle gaps between each launch

Megakernel

one long-running program does the whole model, picking tasks from a queue and fetching the next layer while working

Two engines: llama.cpp, then flashlet

llama.cpp

Our baseline (December)

- Ready-made, widely used
- Runs compressed (4-bit) models
- Lets us choose which layers sit on the GPU
- Gives honest first numbers

flashlet daemon

Engine built this semester; daemon Jan–Apr

- Always-on background server: no waiting to load
- Paged conversation memory, many requests at once
- Megakernel for fast single-student answers
- Same API, so tw-agent just switches URL

Fair comparison: same laptop, same model and same exercises on both engines — we compare words per second, time to first word and memory (experiments E3, E4).

Teaching the model our course

Manish Biswal · Tech

Course data

Exercises + worked solutions; test problems kept separate

Step 1: SFT

Learn from examples. LoRA adds a small trainable add-on instead of changing the whole model

Step 2: GRPO

Try 8 answers per exercise, run the tests, learn more from the answers that scored better

Reward = tests passed ÷ total tests (0 if it crashes or times out)

No human grading needed: the tests are the teacher. Test problems are never used for training, so the final score is fair.

Plan and conclusion

How we will measure success · timeline · where we stand

How we will measure success



Correct

pass@1: share of exercises solved on the first try



Fast

time to first word, and words per second



Fits

peak GPU and RAM memory on the 6 GB laptop

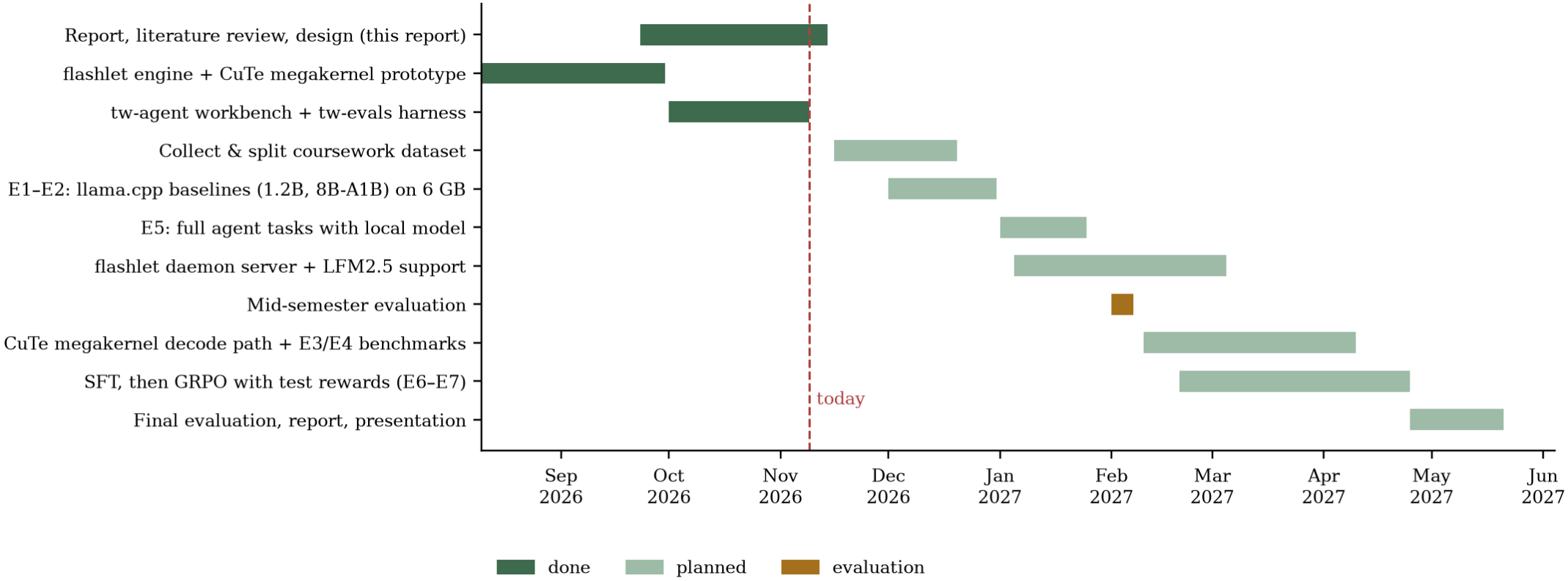


Cheap

₹ per 100 exercises: electricity vs paid API

Fair-test rules: same exercises and settings for every system · failures are recorded, not hidden · every number saved with its exact setup · compared against a hosted model

Timeline


Report, literature review, design (this report)
flashlet engine + CuTe megakernel prototype
tw-agent workbench + tw-evals harness
Collect & split coursework dataset
E1–E2: llama.cpp baselines (1.2B, 8B-A1B) on 6 GB
E5: full agent tasks with local model
flashlet daemon server + LFM2.5 support
Mid-semester evaluation
CuTe megakernel decode path + E3/E4 benchmarks
SFT, then GRPO with test rewards (E6–E7)
Final evaluation, report, presentation
today
Sep 2026
Oct 2026
Nov 2026
Dec 2026
Jan 2027
Feb 2027
Mar 2027
Apr 2027
May 2027
Jun 2027
done
planned
evaluation

Where we stand

Done this semester

- Research questions, literature review and design
- flashlet engine + megakernel (140 tok/s)
- tw-agent workbench, tested end to end
- tw-evals examiner, verified (166/166)
- First pipeline checks: 6/6 and 10/10
- Detailed plan for every experiment

Next semester

- Local baselines with llama.cpp (Dec)
- Full agent tasks on the laptop (Jan)
- flashlet daemon + megakernel (Jan–Apr)
- Course training: SFT then GRPO (Feb–Apr)
- Final comparison: quality, speed, memory, cost

Thank you

Questions welcome

github.com/iamanishx/flashlet · tw-agent · tw-evals